МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
КАЗАНСКИЙ ГОСУДАРСТВЕННЫЙ
ЭНЕРГЕТИЧЕСКИЙ УНИВЕРСИТЕТ

ОТЧЁТ ПО ПРОЕКТНОМУ ПРАКТИКУМУ

DigitLab

Разработка Web приложения и API для распознавания рукописных цифр на основе предварительно обученной модели с Hugging Face

Выполнил: Пескишев Егор Юрьевич
Группа: ЗИИНм-1-26
Форма выполнения: индивидуальная

Казань 2026

Содержание

Введение
1 Обзорная часть
1.1 Обзор задачи
1.2 Предварительно обученная модель
1.3 Инструменты разработки
2 Разработка приложения искусственного интеллекта
2.1 Функции приложения
2.2 Архитектура
2.3 Реализация
3 Организация разработки
3.1 Репозиторий
3.2 Тестирование системы
3.3 Непрерывная интеграция
3.4 Развёртывание в облаке
3.5 Проверка приложения
Заключение
Список источников
Приложение с ключевыми фрагментами кода

Введение

Цель работы — разработать приложение, которое предоставляет доступ к предварительно обученной модели машинного обучения через Web интерфейс и HTTP API. Для демонстрации выбран сценарий распознавания одной рукописной цифры от 0 до 9. Он позволяет проверить полный путь от пользовательского изображения до результата инференса, включая предобработку, обработку ошибок и автоматические тесты.

Проект DigitLab использует готовую модель onnxmodelzoo/mnist-12, размещённую на платформе Hugging Face. Обучение и изменение весов в проекте не выполняются. Основной результат разработки — программная оболочка вокруг модели: API на FastAPI, интерфейс на Streamlit, модуль обработки изображений, тесты и конфигурация развёртывания.

Работа выполняется индивидуально. Все программные компоненты и документация относятся к одному проекту. Требование курса о командах из двух-трёх участников сохраняется как организационное условие, которое необходимо согласовать с преподавателем при индивидуальной сдаче.

1 Обзорная часть

1.1 Обзор задачи

Распознавание цифры является задачей классификации изображения по десяти классам. В учебном сценарии пользователь подаёт изображение одной цифры на однотонном фоне. Приложение возвращает номер класса и оценки для остальных вариантов. Практический смысл работы состоит в организации воспроизводимого сервиса инференса, а не в исследовании новых архитектур нейронных сетей.

1.2 Предварительно обученная модель

Выбрана модель MNIST-12 в формате ONNX из репозитория onnxmodelzoo/mnist-12 на Hugging Face. Согласно карточке модели, свёрточная нейросеть обучена на MNIST в CNTK и использует чередующиеся свёрточные слои и слои max pooling. Вход — float32 тензор формы 1×1×28×28. Выход — десять значений до softmax. Модель обрабатывает одно изображение за вызов.

Файл модели занимает 26 143 байта. Для воспроизводимости загрузчик использует ревизию 1cd752f4a3d818fa215aa621c78b8a10a4a1a3a5. SHA256 сверяется с файлом artifacts/model.sha256. Если содержимое отличается, загрузка завершается ошибкой. Таким образом, изменение внешнего файла не приводит к незаметной подмене весов.

1.3 Инструменты разработки

Streamlit предоставляет элементы загрузки файла, выбор примера и отображение вероятностей. FastAPI реализует HTTP API и интерактивную документацию OpenAPI. ONNX Runtime выполняет модель на процессоре. Pillow и NumPy обеспечивают чтение и нормализацию изображения. pytest и TestClient проверяют API без отдельного внешнего сервера. GitHub Actions описывает запуск тестов при изменении репозитория.

2 Разработка приложения искусственного интеллекта

2.1 Функции приложения

Пользователь может выбрать один из десяти тестовых примеров MNIST или загрузить PNG, JPEG либо WEBP. Ограничение размера файла составляет 5 МБ, количества пикселей — 16 миллионов. После нажатия кнопки приложение передаёт файл в API и показывает распознанную цифру, максимальную оценку softmax и распределение по десяти классам. Ошибки загрузки и недоступность сервера отображаются отдельными сообщениями.

2.2 Архитектура

Клиентский браузер взаимодействует со Streamlit. Серверная часть Streamlit отправляет multipart запрос POST /predict в FastAPI. API читает файл с ограничением размера и запускает предобработку и инференс в пуле потоков, чтобы не выполнять вычисления непосредственно в цикле асинхронного обработчика. Сессия ONNX Runtime создаётся при запуске и повторно используется для запросов.

Разделение интерфейса и API позволяет использовать модель из других клиентов. GET /health подтверждает доступность загруженной модели. /docs открывает документацию Swagger. Предсказание возвращается как JSON с полями digit, confidence, probabilities, model и warning. Приложение не записывает пользовательские изображения на диск.

2.3 Реализация

Изображение преобразуется в оттенки серого. Прозрачность накладывается на белый фон. По граничным пикселям определяется полярность, затем выделяется область штриха. Цифра вписывается в прямоугольник 20×20, помещается на поле 28×28 и центрируется по центру масс. Значения приводятся к диапазону от 0 до 1 и типу float32. Выходные логиты нормализуются устойчивым softmax с вычитанием максимума.

Файлы, которые не удаётся декодировать, и изображения без различимого штриха вызывают ответ 422. Превышение лимита байтов вызывает 413. Эти проверки уменьшают число ошибочных вызовов модели, но не заменяют авторизацию и ограничения частоты запросов для промышленного сервиса.

3 Организация разработки

3.1 Репозиторий с кодом

Автор проекта — Пескишев Егор Юрьевич, группа ЗИИНм-1-26. Профиль GitHub: https://github.com/PoM1nkey. Исходный код содержит интерфейс app.py, пакет digitlab, каталог tests, скрипты подготовки и оценки, notebook Colab и Dockerfile. Файл .gitignore исключает виртуальное окружение, кэши и секреты. Репозиторий и ссылки на выполненные Actions должны быть зафиксированы после фактической публикации.

3.2 Тестирование системы

Автотесты проверяют работоспособность модели через HTTP API, формат и сумму вероятностей, распознавание десяти фиксированных примеров, одинаковую обработку светлого и тёмного фона, отсутствие файла, повреждённые данные, пустое изображение и превышение лимита. Тесты используют настоящие веса ONNX, поэтому подтверждают совместимость модели с обработчиком API, а не только поведение имитации модели.

3.3 Непрерывная интеграция

Workflow GitHub Actions срабатывает на push и pull request. Задание test устанавливает зависимости, загружает закреплённую модель, выполняет pytest и сохраняет JUnit XML как артефакт. Задание deploy зависит от успешного test и может вызвать deploy hook при push в main. Для этого требуется настройка секрета DEPLOY_HOOK. При отсутствии секрета развёртывание явно пропускается.

3.4 Развёртывание приложения в облаке

Dockerfile собирает образ на Python 3.11, устанавливает зависимости и загружает модель. Скрипт serve.py запускает FastAPI на порту 8000 и Streamlit на порту 7860. Образ подходит для Docker хостинга. В Hugging Face Docker Space внешний доступ к порту 7860 предоставляет Web интерфейс, а запросы к API выполняются внутри контейнера. Отдельная публичная публикация API требует соответствующей конфигурации хостинга.

На момент подготовки этой версии отчёта описана и локально проверяется конфигурация. Облачный URL, успешный запуск GitHub Actions и выполнение notebook в Google Colab следует подтвердить после действий в аккаунтах. Наличие workflow или Dockerfile само по себе не доказывает выполнение этих этапов.

3.5 Проверка приложения

Проведена отдельная оценка на первых 1000 изображениях тестовой выборки MNIST в исходном порядке. На вход исходной модели подавались нормированные изображения 28×28 без дополнительного выделения цифры. Точность исходной модели составила 98.4%. При обработке тех же изображений через полный конвейер PNG и предобработки точность составила 98.0%. Результаты сохранены в artifacts/evaluation.json и воспроизводятся скриптом scripts/evaluate.py.

Эта проверка не является оценкой на фотографиях реальных документов. Дополнительное выделение и центрирование изменяет входное распределение, поэтому значения исходной модели и полного конвейера приведены отдельно. Примеры для демонстрации выбраны по первой встреченной метке каждого класса в MNIST test, без отбора по результату предсказания.

Ручная демонстрация включает выбор примера, вызов API и просмотр распределения вероятностей. Пользователь также получает возможность загрузить собственный рисунок. Высокая оценка одного класса не гарантирует правильное распознавание, особенно если вход не является цифрой.

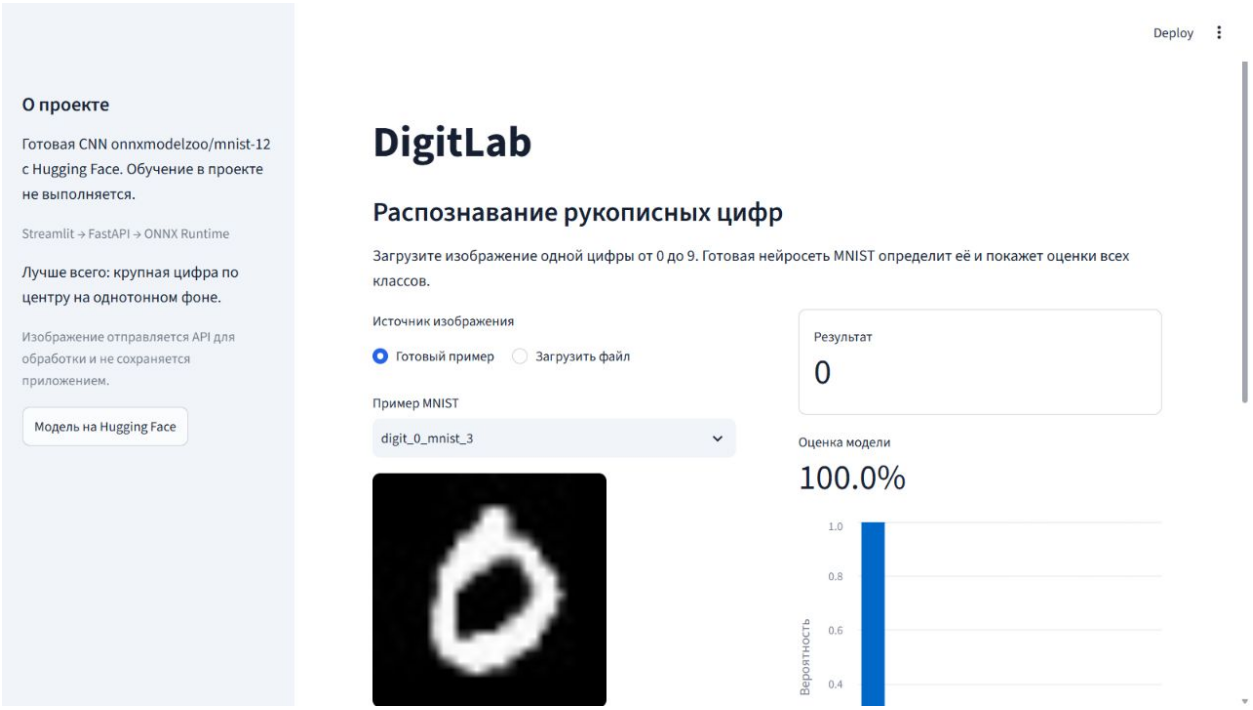


Рисунок 1. Интерфейс DigitLab после обращения к FastAPI

Заключение

В результате разработан проект DigitLab, который предоставляет готовую модель распознавания цифр с Hugging Face через интерфейс Streamlit и API FastAPI. Реализованы нормализация изображения, инференс ONNX, обработка некорректного ввода и автоматические проверки. Подготовлены notebook для демонстрации в Colab, GitHub Actions и Docker конфигурация.

Основные ограничения связаны с областью применения модели. Приложение рассчитано на одну цифру на однотонном фоне, не распознаёт текст и не выделяет отдельные цифры из многозначного числа. Модель не содержит отдельного класса для посторонних изображений. Для дальнейшего развития можно добавить обнаружение неподходящего входа, оценку на пользовательских данных и ограничения частоты запросов.

Для завершения внешних этапов практикума необходимы опубликованный репозиторий, подтверждённый запуск CI, работающий облачный сервис и выполнение notebook в Google Colab. Соответствующие ссылки добавляются по факту публикации.

Список использованных источников

1. КГЭУ. Требования к проектам. https://lms2.kgeu.ru/mod/page/view.php?id=14361
2. КГЭУ. Проект 60 баллов. https://lms2.kgeu.ru/mod/assign/view.php?id=14368
3. Hugging Face. onnxmodelzoo/mnist-12. https://huggingface.co/onnxmodelzoo/mnist-12
4. ONNX Model Zoo. MNIST. https://github.com/onnx/models/tree/main/validated/vision/classification/mnist
5. Тестовые данные MNIST. https://storage.googleapis.com/cvdf-datasets/mnist/
6. FastAPI. Документация. https://fastapi.tiangolo.com/
7. Streamlit. Документация. https://docs.streamlit.io/
8. GitHub Actions. Документация. https://docs.github.com/actions

Дата обращения к требованиям и модели: 24 октября 2026 года.

В метаданных модели на Hugging Face указана лицензия Apache-2.0, а в теле карточки и исходном ONNX Model Zoo указана MIT. Это расхождение зафиксировано в THIRD_PARTY_NOTICES.md; проект не изменяет лицензионные обозначения авторов модели.

Приложение 1 Ключевые фрагменты кода

Запуск инференса и нормализация результата

```
runtime = session()
logits = runtime.run(
    None,
    {runtime.get_inputs()[0].name: tensor}
)[0].reshape(-1)
scores = np.exp(logits - logits.max())
probabilities = scores / scores.sum()
digit = int(probabilities.argmax())
```

Последовательность воспроизведения

```
python -m pip install -r requirements-dev.txt
python scripts/download_model.py
python -m pytest -q
python scripts/evaluate.py
python scripts/serve.py
```

Конфигурация CI

Файл .github/workflows/ci.yml запускает тесты на push и pull request. Модель загружается до pytest. Задание deploy зависит от test и использует секрет DEPLOY_HOOK. Полный код поставляется вместе с отчётом.